

오늘의 작업기록 — 2026년 7월 8일

라이트 데이 — 복기 문답집 제작 + 잔디 정리

(m105를 담은 다음 날, 숨 고르기가 계획의 일부인 날)

한눈에

항목	내용
성격	라이트 데이 (계획된 감속) — 새 개념 0, 서버 안 띄움
결과	① mini-ledger .gitignore 정리(커밋 3480b61) ② 복기 문답집 Q1~Q53 완성 (660줄 md + A4 PDF)
오늘의 판단	복기 방식 전환: 빈 화면 재현 → 인쇄용 심화 문답집 (재현은 부록 체크리스트로 연결)
쉬어도 되는 근거	관문 1(7/31) 조건 7/5 조기 충족 · 7월 잔여 = m106+m107(2~3세션/24일) · 6/19부터 19일 연속 가동
오늘의 개념 한 톨	.gitignore에서 --cached가 필요한 경우 vs 불필요한 경우 — 갈림 기준은 "이미 추적 중이냐"
다음	재진입점 = 예외→HTTP 번역(잔액 부족이 500으로 새는 것) · m106은 7월 후반 · 원자성/checkBalance 빈 화면 재현 예약 유지

어제 m105를 담았다 — 같은 키 두 번에 잔액 불변, 멍등성이 HTTP에서 실전 검증된 큰 마일스톤. 오늘은 그 다음 날이고, 컨텍스트에 직접 박아둔 원칙이 있다: "9개월은 스프린트가 아니라 마라톤. 갈아넣되 지속 가능하게." 마일스톤을 담은 다음 날이 정확히 숨 고르는 자리다. 쉬는 게 이탤이 아니라 계획의 일부라는 것 — 숫자가 그걸 뒷받침한다(관문 조기 충족, 19일 연속). 그래서 오늘은 잔디 5분 + 복기만. 대신 복기의 방식을 하나 설계 변경했고, 그게 오늘의 실질 산출물이다.

0. 오늘의 위치 — 왜 감속이 "계획"인가

- 관문 1(7/31) 조건 = Java 기초 + mini-ledger 완성 → 7/5에 이미 충족. 3주 반 조기 달성 상태.
- 7월에 남은 것: m106(CodeCrafters Redis 1단계, 무게 2) + m107(월말 점검) — 합쳐 2~3세션짜리를 24일 안에 하면 된다.
- 6/19 시작 이후 19일 연속 가동. 지속 가능성이 곧 완주 조건이다.
- 라이트 데이 설계: 잔디 5분 + 복기 30~40분. 새 개념 0, 서버 안 띄움. (진짜 0으로 쉬는 선택지도 유효했다 — 잔디 하루 비는 것만 감수하면. 오늘은 예약된 복기가 있어 라이트 데이를 골랐다.)

1. 잔디 5분 — mini-ledger IDE 부산물 정리

7/5에 적어둔 정리 항목을 오늘 털었다.

```
cd ~/mini-ledger
printf "out/\n.idea/\n*.iml\n" >> .gitignore
git add . && git commit -m "ignore IDE artifacts" && git push
```

판정:

```
[main 3480b61] ignore IDE artifacts
1 file changed, 3 insertions(+)
96f828d..3480b61  main -> main
```

세 항목의 정제 (왜 추적에서 빼나):

항목	정제	제외 이유
out/	IntelliJ가 만드는 컴파일 산출물 폴더	소스에서 언제나 재생성 — 결과물이지만 원본이 아님
.idea/	IntelliJ 프로젝트 설정 폴더	내 PC 환경에 종속 — 다른 환경에선 무의미/충돌
*.iml	IntelliJ 모듈 파일	위와 동일 — IDE 전용 메타데이터

공통 원칙: repo에는 소스(원본)만. 도구가 만들어내는 것·환경에 종속된 것은 제외. (server.log를 뺀 것과 같은 원칙의 연장이다 — 그건 "실행이 만들어내는 것"이었다.)

★ 오늘의 개념 한 톨 — 어제는 --cached가 필요했는데, 오늘은 왜 ignore만으로 끝났나

같은 .gitignore 작업인데 절차가 달랐다:

- 어제(7/7) server.log: .gitignore 추가 + git rm --cached 두 단계 필요 — 이미 커밋돼 추적 중이었기 때문. ignore는 추적 중인 파일엔 무력하다.
- 오늘 out/.idea/*.iml: .gitignore 추가 한 단계로 끝 — 이 항목들은 mini-ledger repo에 커밋된 적이 없다(push를 VPS에서만 했고, IDE 부산물은 사지방 PC 로컬에만 생겼으므로). 추적 목록에 없는 파일은 ignore가 바로 먹는다.

갈림 기준 한 줄: "그 파일이 이미 추적 중이냐." 추적 중이면 2단계(ignore + rm --cached), 아니면 1단계(ignore만). 판정이 필요하다면 git ls-files | grep 파일명 으로

추적 여부를 직접 확인할 수 있다. — 같은 도구를 이를 연속 다른 상황에서 쓰면서, 어제의 개념(--cached의 존재 이유)이 오히려 단단해졌다.

(부수 확인: 이 커밋은 7/5 이메일 통일 이후 mini-ledger에서 찍은 **첫 커밋** — noreply 체계에서 잔디가 정상적으로 이어지는 것도 검사검사 확인된 셈이다.)

2. 오늘의 핵심 판단 — 복기 방식의 전환

원래 예약(7/8~9): 빈 화면 재현 — 원자성 트레이스(a=1010 버그 버전) + checkBalance를 기억만으로 재작성.

전환한 것: 오늘 하루는 **인쇄용 심화 문답집** 방식으로. 7/6~7/7 전 과정을 질문·모범답안(Q&A)으로 문서화하고, A4로 인쇄해 정독·셀프테스트한다.

전환 이유: 복기 대상이 달랐다 — 예약돼 있던 재현 과제는 7/5분(원자성·Map)이었는데, 지금 굳혀야 할 것은 **7/6~7/7의 스프링 전환**(record, @RequestBody, 상태를 가진 서버 등 첫 등장 개념 덩어리)이다. 전환기 개념은 "손으로 재현"보다 먼저 **구조화된 설명을 정독하며 이해를 재구축**하는 단계가 맞다고 판단했다. 인쇄물은 화면 없이(생활관에 서도) 반복 가능하다는 실용 이점도 있다.

두 방식의 성격 — 상보적이다:

	빈 화면 재현	문답집 정독/셀프테스트
훈련 방향	출력(꺼내기) — 손이 기억하나	입력+구조화 — 이해가 서 있나
잘 잡는 것	문법·코드의 근육 기억	개념 사이의 연결·"왜"
실패 신호	손이 멈추는 지점	답을 잃어도 가뭇한 지점

그래서 **버린 게 아니라 연결했다:** 문답집 **부록에 "빈 화면 재현 체크리스트 8개"**를 넣어, 정독이 끝나면 인쇄물을 덮고 재현으로 넘어가게 설계했다. 그리고 7/5분 재현(원자성 트레이스 + checkBalance)은 **예약 그대로 유지** — 문답집 말미에 다음 복기 후보로 명시해뒀다.

3. 산출물 — 복기 문답집 (Q1~Q53 + 부록)

파일: 복기문답_스프링전환_0706-0707.md (660줄, 48KB) + .pdf (A4 인쇄용, 256KB) **배치:** 대시보드 logs/ 에는 넣지 않음 — 일일 작업기록이 아니라 학습 교재이므로, logs/의 성격(그날 무엇을 했나)을 흐리지 않는다.

구성 (13개 PART, 53문항):

- **PART 1** 큰 그림 — 서버 vs 일회성 프로세스, ④호출자만 교체된 아키텍처, 마법 목록 4개
- **PART 2~6 [7/6]** — start.spring.io 옵션 해부 / 생성 파일 역할 / package·애노테이션·@GetMapping / Gradle과 16분의 정체·죽지 않는 프로세스 / curl -v HTTP 해부·개행 함정·ss / push≠잔디와 login 판정 / "동작 확인→커밋"
- **PART 7~11 [7/7]** — git rm --cached / sed '1i' / BUILD SUCCESSFUL의 한계 / **record**(대체하는 손코드 20줄 병기)·DTO / GET vs POST·**@RequestBody 4단계**·curl 플래그 / ★**상태를 가진 서버**(수명 도식, Main 대조표, key-1 2차 메커니즘, 영속성 부재) / 검증 3연타 표·**"돈 보존 ×2" 구조 증명**·Stripe·토스 계약 / 다음 조각 (500→400대) 예고
- **PART 12** 트러블 4건 — 404(파일 부재), 부모·자식 프로세스, IntelliJ 먹통 판단, update.py --done 원칙
- **PART 13** 관통 — **착시 5개 × 판정 도구** 표, 8월 WAL로 수렴하는 두 실(원자성 한계 + 영속성 부재)
- **부록** — 인쇄물을 덮고 하는 **빈 화면 재현 체크 8개**

제작 파이프라인(재현용): markdown → python-markdown으로 HTML 변환(tables·fenced_code 확장) → A4 인쇄 CSS(Noto Sans CJK, 페이지 여백, 코드블록 페이지 분리 방지) → wkhtmltopdf. 일일 로그의 make-pdf.py(D-day 헤더 포함)와 별도 파이프라인을 쓴 이유 = 이 문서는 날짜 로그가 아니라서.

4. 로드맵 위치와 다음

- **m103 · m105 done.** 관문 1은 사실상 달성 상태 — m107(월말 점검)에서 정식 선언만 남음.
- **다음 온전한 세션 재진입점 = 예외→HTTP 번역.** 잔액 부족(클라이언트 사정)이 500(서버 탓)으로 새는 것을 400대로 바로잡기. 지금 a=60 상태라 key-3 이체 한 방이면 바로 관찰 가능 — 진입 비용이 낮게 세팅돼 있다.
- **m106(CodeCrafters Redis)** — 7월 후반 슬롯.
- **복기 잔여 예약:** ① 문답집 부록 체크 8개(인쇄물 공부 후) ② 원자성 트레이스 + checkBalance 빈 화면 재현(7/5분).

5. 검증 질문 (다음 복기용 — 답 적지 말 것)

1. 어제 server.log는 2단계(ignore + --cached)였는데 오늘 out/ 등은 1단계로 끝났다 — 갈림 기준은? 추적 여부를 직접 판정하는 명령은?
2. out/ · .idea/ · *.iml은 각각 무엇이고, 공통 제외 원칙은 한 문장으로?
3. 오늘의 라이트 데이가 "이탈이 아니라 계획"인 근거를 숫자 셋으로 대라.
4. 복기의 두 방식(재현 vs 문답 정독)은 각각 무엇을 훈련하고, 오늘 둘을 어떻게 연결했나?
5. 복기문답집을 대시보드 logs/에 넣지 않은 이유는?
6. 다음 온전한 세션의 재진입점과, 그 진입 비용이 낮은 이유(현재 계좌 상태)는?

6. 회고

19일 만에 처음 감속했다. 어색했지만 — 원칙을 박아둔 이유가 정확히 이런 날을 위해서였다. "마일스톤을 담은 다음 날이 숨 고르는 자리"라는 문장은 관문 조기 총족이라는 숫자가 있어야 힘을 갖고, 그 숫자는 6월부터의 연속 가동이 만들었다. 씬이 계획의 일부가 되려면 그 앞의 달림이 계획대로였어야 한다는 것 — 오늘 그게 맞물렸다.

실질 산출물은 문답집이다. 복기 방식을 스스로 설계 변경했다는 점이 코드 한 줄보다 남는다 — "어떻게 공부할 것인가" 자체를 도구로 만들었다. 전환기 개념 덩어리(record, 객슨, 상태를 가진 서버)는 손 재현보다 구조화된 정독이 먼저라고 판단했고, 대신 재현을 버리지 않고 부록 체크리스트로 이어 붙였다. 인쇄물 한 부로 정독→덮고 재현까지 한 사이

클이 되게 — 학습 파이프라인을 짠 셈이다.

그리고 작지만 오늘의 개념 한 톨: 같은 .gitignore인데 어제는 두 단계, 오늘은 한 단계였다. "이미 추적 중이냐"라는 갈림 기준은 어제 --cached를 배울 때보다 오늘 그것이 **불필요한 경우**를 만나면서 오히려 선명해졌다. 개념은 쓰일 때가 아니라 안 쓰일 때를 구분할 수 있을 때 내 것이 된다.

내일부터의 길은 명확하다. 재진입점(예외→HTTP 번역)은 어제 위치를 찍어뒀고, 진입 비용까지 낮다(a=60, curl 한 방식이면 500이 관찰된다). 오늘 고른 숨이 그 진입을 가볍게 만들 것이다.

한 줄: 계획대로 감속한 날 — 잔디 5분(ignore 한 단계로 충분한 경우를 배움) + 7/6~7/7 전 과정을 Q53 인쇄용 문답집으로 구조화. 재현은 부록으로 연결, 재진입점은 예외→HTTP 번역.